Sanjivani Rural Education Society's
Sanjivani College of Engineering, Kopargaon-423 603
(An Autonomous Institute, Affiliated to Savitribai Phule Pune University, Pune)
NAAC 'A' Grade Accredited, ISO 9001:2015 Certified

Department of Computer Engineering
(NBA Accredited)

Subject- Object Oriented Programming (CO212)
Unit 2 – Overloading and Inheritance
Topic – 2.5 Public and Private Inheritance

Prof. V.N.Nirgude
Assistant Professor
E-mail :
nirgudevikascomp@sanjivani.org.in
Contact No: 9975240215

Access Control

- **Access Specifier and their scope**

Base Class Access Mode	Derived Class Access Modes		
	Private derivation	Public derivation	Protected derivation
Public	Private	Public	Protected
Private	Not inherited	Not inherited	Not inherited
Protected	private	Protected	Protected

Public Derivation

- By deriving a class as public, the public members of the base class remains public members of the derived class and protected members remain protected members.

Class A

```
private :
        int a1;

protected :
      int a2;

public :
      int a3;
```

Class B

```
private :
      int b1;

protected :
      int b2;

public :
       int  b3;
```

Class B: Public A

```
private:
   int b1;

protected:
 int a2;
 int b2

public:
int b3;int a3;
```

Example using public derivation

```
class Rectangle
{
    private:
       float length ;
    public:
       float breadth ;
       void Enter_lb( )
       {
         cout << "Enter length of the rectangle : ";
         cin >> length ;
         cout << "Enter breadth of the rectangle : ";
         cin >> breadth ;
    }
    float Enter_l( )
    {
        return length ;
    }

    };
    class Rectangle1 : public Rectangle
    {
         private:
            float area ;
    public:
        void Rec_area()
        {
            area = Enter_l( ) * breadth ;
        }

        void Display()
        {
            cout << "Length = " << Enter_l( ) ;
            cout << "Breadth = " << breadth ;
            cout << "Area = " << area ;
        }
};
void main()
{
     Rectangle1 r1 ;
     r1.Enter_lb( );
     r1.Rec_area( );
     r1.Display( );
}
```

Private Derivation

- If we use private visibility mode while deriving a class then both the public and protected members of the base class are inherited as private members of the derived class.

Class A

```
private :
    int a1;

protected :
    int a2;

public :
    int a3;
```

Class B

```
private :
    int b1;

protected :
    int b2;

public :
    int b3;
```

Class B : private A

```
private :
int b1;
int a2,a3;

protected:
int b2;

public:
int b3;
```

Example using private derivation

```
class Rectangle
{
        int length, breadth;
    public:
    void enter()
    {
            cout << "Enter length and breadth: ";
            cin >>  length>>breath;
    }
    int getLength()
    {
            return length;
    }
    int getBreadth()
    {
            return breadth;
    }
    void display()
    {
            cout << "Length= " <<length;
            cout << "Breadth= " <<breadth;
    }
    };
```

```
class RecArea : private Rectangle
{

public:
void area_rec()
{
            enter();
            cout << "Area = " << (getLength()
*getBreadth());
}
};

int main()
{
            RecArea r ;
            r.area_rec();
            return 0;
}
```

Output:

```
Enter length and breadth: 10 20
Area = 200
```

Protected Derivation

- It makes the derived class to inherit the protected and public members of the base class as protected members of the derived class.

Class A

```
private :
    int a1;

protected :
    int a2;

public :
    int a3;
```

Class B

```
private :
    int b1;

protected :
    int b2;

public :
    int b3;
```

Class B : Protected A

```
private :
    int b1;

protected:
int a2;
int b2,a3;

public:
int b3;
```

Example using protected derivation

```
class GrandParent
{
   public:
   void grandParentMethod()
   {
            cout<<"Method in the grand parent class";
   }
};
class Parent : protected GrandParent
{
   public:
   void parentMethod()
   {
            cout<<"Method in the parent class";
   }
};
class Child: protected Parent
{
   public:
   void childMethod()
   {
            cout<<"Method in the child class";
            parentMethod();
            grandParentMethod();
   }
};
int main()
{
   Child C;
   C.childMethod();
 }
```

Output:

Method in the child class
Method in the parent class
Method in the grand parent class


S.R.E.S.
SANJIVANI COLLEGE OF ENGINEERING
ESTD.1983
KOPARGAON

Thank You..